Notații Asimptotice

2023

Cuprins

1 Introducere **1**

2 Obiective **1**

3 Model teoretic **2**

4 Notații asimptotice [1] **2**
4.1 Definirea notațiilor asimptotice folosind limite 4
4.2 Proprietăți 5

5 Exemple **5**

1 Introducere

Fie o problema P, pentru care am identificat mai multe soluții care produc rezultatul corect pentru orice intrare. Cum putem identifica cea mai bună soluție?

În această parte a cursului ne propunem să analizăm timpul de execuție, respectiv cantitatea de memorie utilizată. Astfel, o abordare directă pentru a compara doi algoritmi ar fi să executăm programul corespunzător fiecărui algoritm pe o anumită mașină de calcul, respectiv același set de teste. Întâmpinăm însă câteva probleme cu această abordare:

- Pentru ca rezultatele să fie relevante, avem nevoie să construim **date de test reprezentative** pentru situațiile pe care le putem întâlni în practică.
  - În funcție de aplicație am putea fi interesați în mod special să evaluăm algoritmii pe cel mai dezvantajos tip de teste.
- **Rezultatele depind de mediul de execuție** (compilator, sistem de operare, arhitectura calculatorului, etc.).
- Întregul proces poate fi foarte costisitor.

Astfel, ne propunem să investigăm o soluție pentru a descrie robust evoluția resurselor consumate pornind doar de la specificația abstractă a algoritmilor.

2 Obiective

În acest curs ne propunem să:

- Introducem conceptul de notații asimptotice și proprietățile acestora.
- Exemplificăm cum putem modela resursele consumate de algoritm (timp, spațiu) folosind notațiile asimptotice.

3 Model teoretic

Pentru a modela matematic resursele consumate de algoritm, în particular timpul de execuție, vom face următoarele presupuneri:

- O instrucțiune de un anumit tip (e.g. "a + b") necesită întotdeauna același timp, indiferent de momentul de execuție[1]. Mai mult, presupunem că putem descompune orice expresie din descrierea algoritmului într-o secvență de operații elementare ($+$, $-$, $*$, $/$, $<$, $>$, ... etc.).
  - Vom considera că **fiecare operație elementară se execută în același timp**.
  - Practic, vom număra doar câte operații elementare s-ar executa pe un anumit tip de test.
- Putem modela resursele consumate ca o funcție monoton crescătoare care depinde **doar de dimensiunea datelor de intrare**.
  - Fie $T : \mathbb{N} \to \mathbb{N}$, funcția care descrie numărul de operații elementare executate de algoritm în funcție de dimensiunea datelor de intrare, dat fiind un algorithm care admite o singură intrare de dimensiune variabilă.
  - Această presupunere este cea mai restrictivă, deoarece, **în funcție de valoarea intrărilor, algoritmul analizat poate să aibă un comportament foarte diferit**. Prin urmare, vom analiza separat performanța algoritmilor pe cel mai dezavantajos tip de date de intrare, cel mai avantajos, respectiv pe cazul mediu. În ultimul caz vom analiza comportamentul pe toate tipurile posibile de intrări, ponderând resursele consumate pe o anumită intrare cu probabilitatea de a întâlni intrarea respectivă în practică.
- Ne interesează să analizăm în primul rând comportamentul algoritmilor pentru un volumn mare al datelor de intrare.
  - Pentru un volum foarte mic al datelor de intrare, resursele consumate vor fi probabil oricum neglijabile.

Din expresia matematică pe care o vom obține analizând algoritmii:

- **Vom ignora factorii constanți**, care depind și de mediul de execuție.
- Dacă expresia obținută este un polinom, **vom ignora termenii de ordin inferior**, care au un impact mic pentru un volum mare de date.

Astfel, vom încadra comportamentul algoritmilor folosind notațiile asimptotice.

4 Notații asimptotice [1]

Fie $f : \mathbb{N} \to R_+$, o funcție asimptotic crescătoare.

Definiție 1: O - Limită superioară

$O(g(n)) = \{\ h : \mathbb{N} \to R_+ \mid \exists c \in \mathbb{R}_+, n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, h(n) \leq c * g(n)\}$

[1] În practică, acest timp poate să varieze, de exemplu dacă variabilele a sau b se află sau nu în cache.

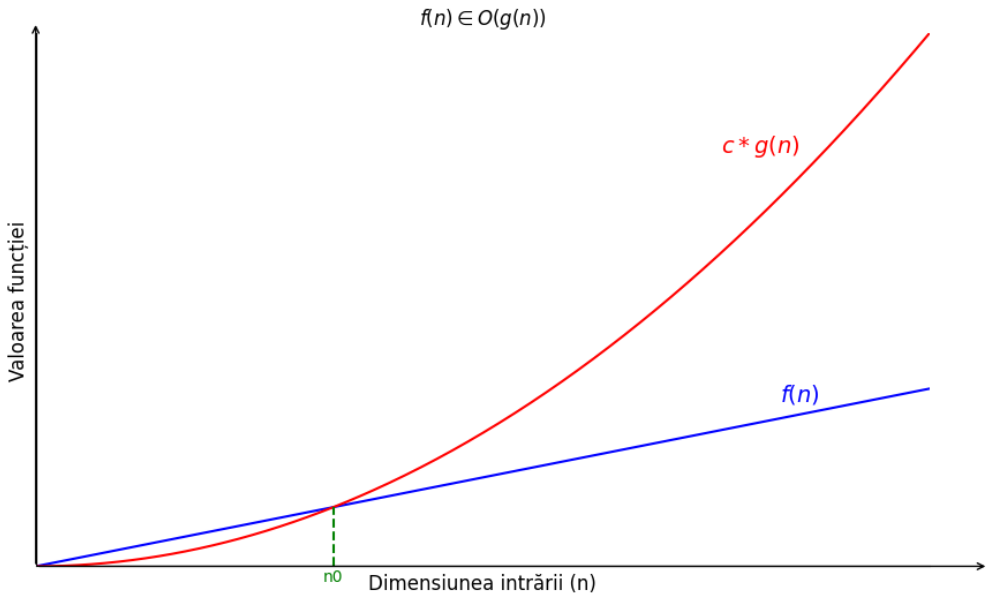


Figura 1: Notăm $f(n) \in O(g(n))$ pentru a exprima faptul că g reprezintă o limită superioară pentru f, pentru valori ale lui n suficient de mari.

Definiție 2: Ω - Limită inferioară

$\Omega(g(n)) = \{\ h : \mathbb{N} \to R_+ \mid \exists c \in \mathbb{R}_+, n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0, c * g(n) \leq h(n)\}$

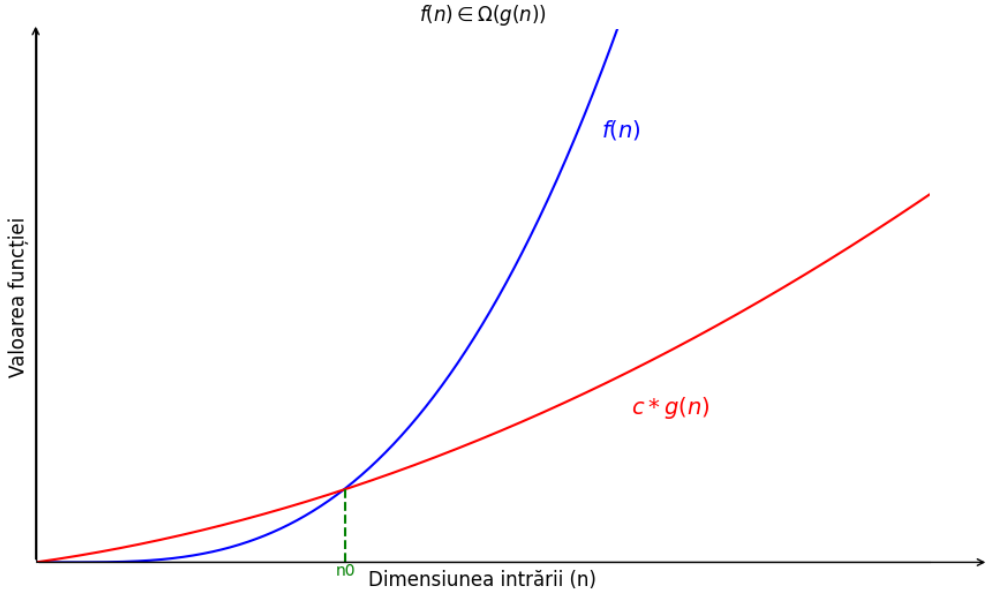


Figura 2: Notăm $f(n) \in \Omega(g(n))$ pentru a exprima faptul că g reprezintă o limită inferioară pentru f, pentru valori ale lui n suficient de mari.

Definiție 3: Θ

$\Theta(g(n)) = \{\ h : \mathbb{N} \to R_+ \mid \exists c_1, c_2 \in \mathbb{R}_+, n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0, c_1 * g(n) \leq h(n) \leq c_2 * g(n)\}$

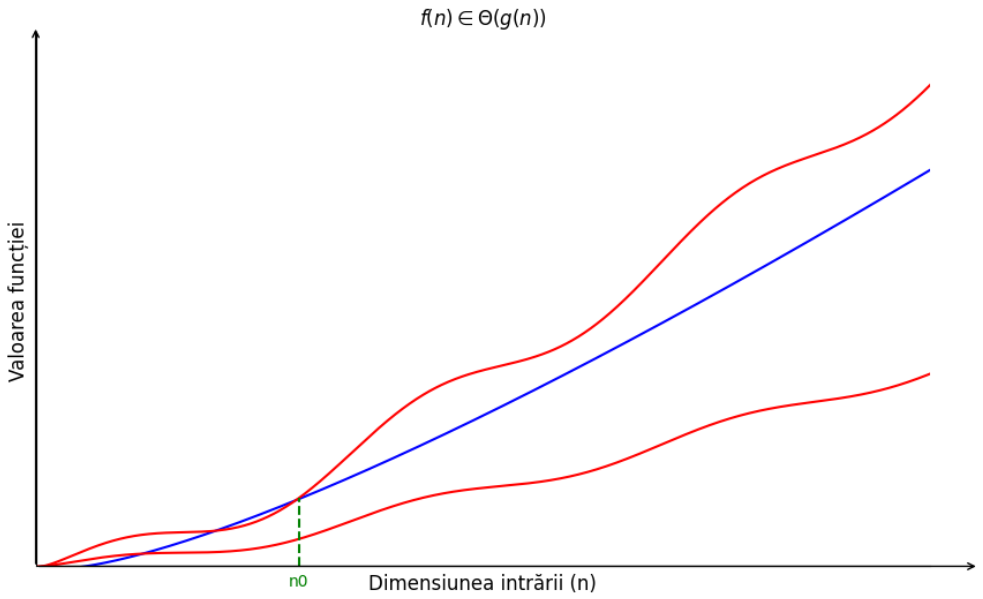


Figura 3: Notăm $f(n) \in \Theta(g(n))$ pentru a exprima faptul că g reprezintă atât o limită inferioară, cât și o limită superioară, pentru valori suficient de mari ale lui n.

Acestea sunt notațiile cele mai des folosite. Mai puteți întâlni:

Definiție 4: o - Limită superioară strictă

$o(g(n)) = \{\ h : \mathbb{N} \to R_+ \mid \forall c \in \mathbb{R}_+, n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, h(n) < c * g(n)\}$

Diferența principală fată de O este faptul că inegalitatea, în acest caz **strictă**, este adevărată pentru **orice** constantă reală pozitivă.

Definiție 5: ω - Limită inferioară strictă

$\omega(g(n)) = \{\ h : \mathbb{N} \to R_+ \mid \forall c \in \mathbb{R}_+, n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, c * g(n) < f(n)\}$

Analog, folosim ω pentru a exprima o inegalitate strictă.

Observație 1: Abuz de notație

Conform definiției, O reprezintă o mulțime de funcții. Cu toate acestea, o putem întâlni, în practică, în expresii precum:

- $f(n) = O(g(n))$, cu semnificația $f(n) \in O(g(n))$
- $f(n) + O(g(n))$ cu semnificația că al doilea termen corespunde unei funcții anonime h din $O(g(n))$.

Motivația acestui abuz de notație este pragmatică; Ne ajută să simplificăm expresiile de calcul pe care le obținem, păstrând doar informația esențială pe care vrem să o transmitem.

4.1 Definirea notațiilor asimptotice folosind limite

Presupunem că există limita $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L$. Atunci:

- $0 \leq L < \infty \to f(n) \in O(g(n))$
- $0 < L \leq \infty \to f(n) \in \Omega(g(n))$
- $0 < L < \infty \to f(n) \in \Theta(g(n))$

- $L = 0 \rightarrow f(n) \in o(g(n))$
- $L = \infty \rightarrow f(n) \in \omega(g(n))$

4.2 Proprietăți

Tranzitivitatea $(O, \Omega, \Theta, o, \omega)$:

- Exemplu: $f(n) \in O(h(n)) \wedge h(n) \in O(g(n)) \Rightarrow f(n) \in O(g(n))$

Reflexivitatea (O, Ω, Θ):

- Exemplu: $f(n) \in O(f(n))$

Simetria:

- $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$

Antisimetria:

- $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$

Altele:

- $f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n))$
- $\Theta(f(n) + g(n)) = \Theta(f(n)) + \Theta(g(n))$
- $\Theta(f(n) * g(n)) = \Theta(f(n)) * \Theta(g(n))$

5 Exemple

Complexitate	Exemple Algoritmi	Problemă
$\theta(1)$	min(a, b)	Minimul dintre două numere.
$\theta(log(n))$	Căutare Binară	Găsirea unui element intr-un șir sortat
$\theta(sqrt(n))$	Sqrt Decomposition	O soluție care să permită aflarea elementului minim între două poziții dintr-o secvență, precum și actualizări la poziții arbitrare.
$\theta(n)$	Median of Medians	Identificarea medianei unei secvențe.
$\theta(n * log(n))$	MergeSort	Sortare
$O(n^2)$	InsertionSort	Sortare
$\theta(n^3)$	Floyd-Warshall	Cel mai scurt drum între oricare două noduri.
$\theta(n^2 * 2^n)$	Held-Karp	Travelling Salesman

References

[1] Charles E Leiserson et al. *Introduction to algorithms*. MIT press, 2009.